```
1 int example(int n) {
2   int x = n * 2;
3   int y = 0;
4   for (unsigned int i = 0; i < n; i++) {
5     y += x + 4 + n;
6   }
7   return y;
8 }
```

Clang 13
(O1)

```
_example:
  push    rbp
  mov     rbp, rsp
  test    edi, edi
  je      LBB0_1
  mov     eax, edi
  add     eax, -1
  lea     ecx, [rdi + 2*rdi]
  add     ecx, 4
  imul    ecx, eax
  lea     eax, [rdi + 2*rdi]
  add     eax, ecx
  add     eax, 4
  pop     rbp
  ret
LBB0_1:
  xor     eax, eax
  pop     rbp
  ret
```

```
1 int example(int n) {
2   int x = n * 2;
3   int y = 0;
4   for (unsigned int i = 0; i < n; i++) {
5     y += x + 4 + n;
6   }
7   return y;
8 }
```

Clang 13
(O1)

6 / 17 instructions mapped to wrong source lines

```
_example:
  push    rbp
  mov     rbp, rsp
  test    edi, edi
  je      LBB0_1
  mov     eax, edi
  add     eax, -1
  lea     ecx, [rdi + 2*rdi]
  add     ecx, 4
  imul    ecx, eax
  lea     eax, [rdi + 2*rdi]
  add     eax, ecx
  add     eax, 4
  pop     rbp
  ret
LBB0_1:
  xor     eax, eax
  pop     rbp
  ret
```

```
1 int example(int n) {
2   int x = n * 2;
3   int y = 0;
4   for (unsigned int i = 0; i < n; i++) {
5     y += x + 4 + n;
6   }
7   return y;
8 }
```

Clang 13
(O1)

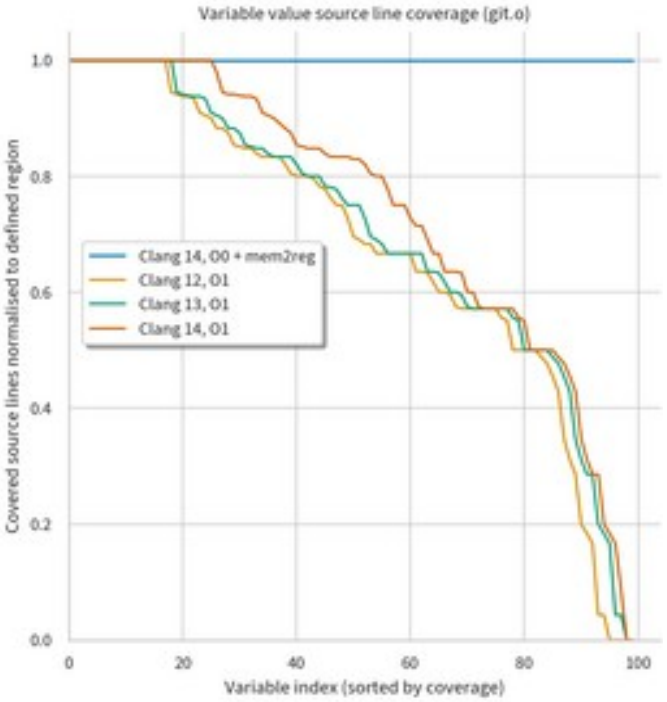
6 / 17 instructions mapped to wrong source lines

Variable y not available for 4 / 17 instructions

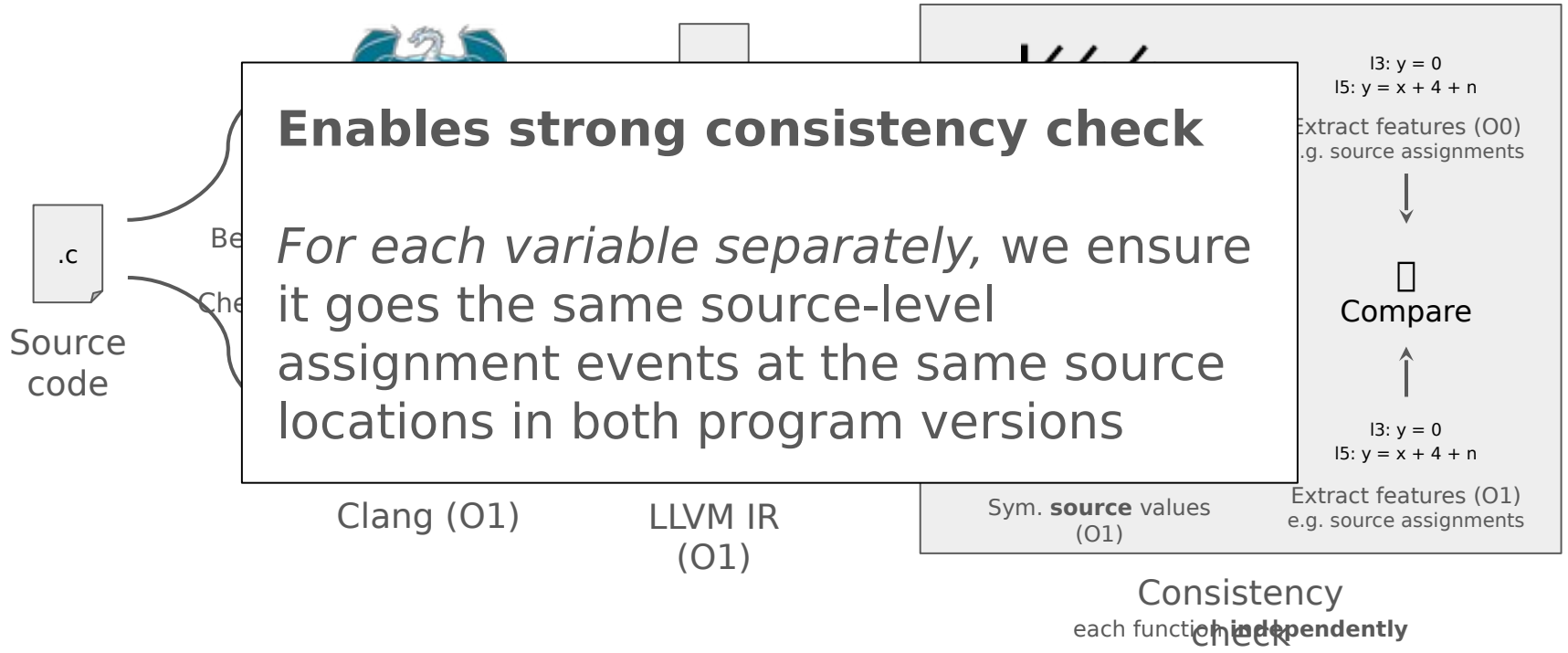
```
_example:
  push    rbp
  mov     rbp, rsp
  test    edi, edi
  je      LBB0_1
  mov     eax, edi
  add     eax, -1
  lea     ecx, [rdi + 2*rdi]
  add     ecx, 4
  imul    ecx, eax
  lea     eax, [rdi + 2*rdi]
  add     eax, ecx
  add     eax, 4
  pop     rbp
  ret
LBB0_1:
  xor     eax, eax
  pop     rbp
  ret
```

Debug info value coverage

- Will talk about coverage metrics in more detail later in the talk...
- In programs optimised by common compilers, most variables are missing values for at least 1 line where they are defined
- Some variables are entirely inaccessible



Debug info consistency check



Disabling optimisation is not always an option

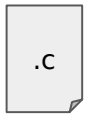
- Real scenarios for optimised debugging
  - Core dumps collected in production
  - Resource heavy programs (e.g. video games) which are too slow without optimisation
  - Programs whose behavior depends on optimisation (e.g. Linux kernel)
  - Tracing unwanted behaviours (e.g. race conditions, memory errors) which may only occur with optimisation
  - **Any program … if you want to debug what actually ran!**
- Poor developer experience has trained many programmers to assume optimised debugging is somehow insurmountable
  - Some may avoid using debuggers entirely
  - In some cases, you can rebuild without optimisation and try debugging again…

Lost in the pipes

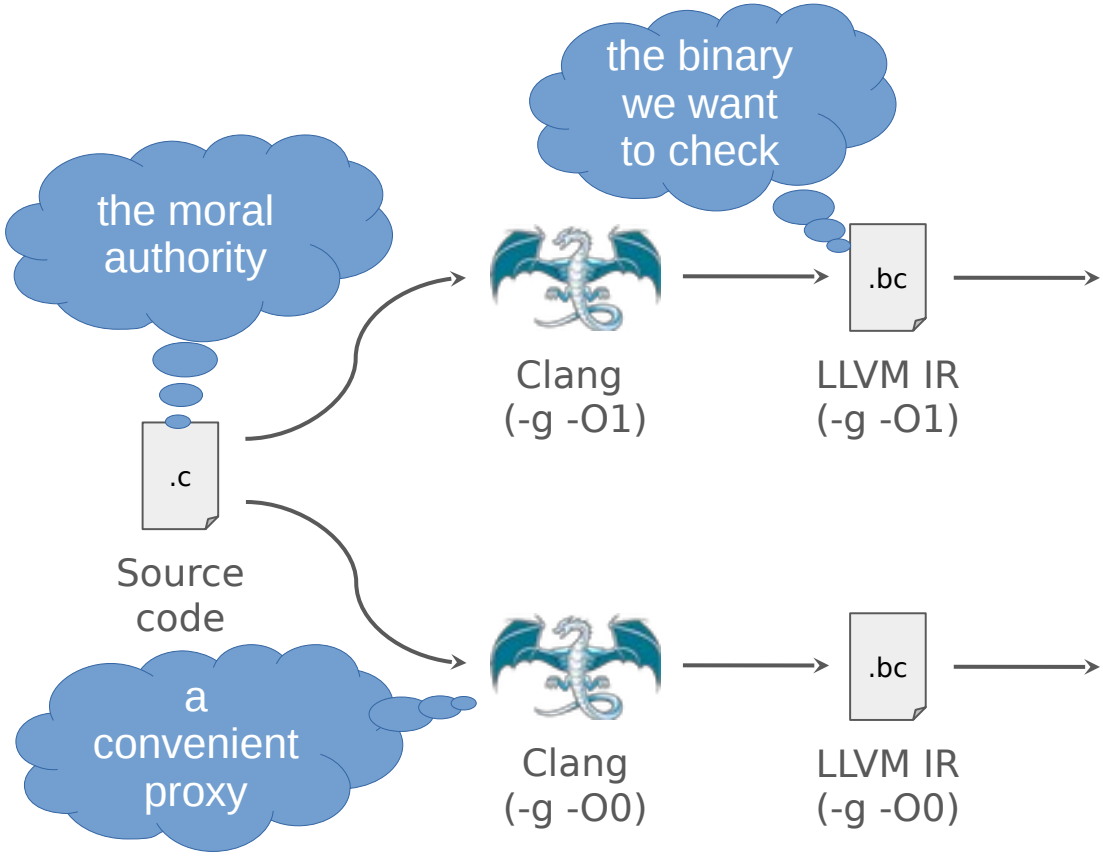
- Optimisations today often corrupt or drop debug info
- Testing debug info is often manual, has poor coverage
  - SN Systems, LLVM contributors. Dexter. 2019.
- Recent work brings some automation, but checks values with heuristics
  - Li et al. Debug information validation for optimized code. PLDI 2020.
  - Di Luna et al. Who's debugging the debuggers? Exposing debug information bugs in optimized binaries. ASPLOS 2021.
  - Assaiante et al. Where did my variable go? Poking holes in incomplete debug information. ASPLOS 2023.
- Stronger testing would help spot more debug info handling bugs
  - Should lead to more reliable debugger experience overall

Approach

Debug info consistency check



Source code


the binary we want to check
the moral authority
.bc
Clang (-g -O1)
LLVM IR (-g -O1)
.c
Source code
.bc
a convenient proxy
Clang (-g -O0)
LLVM IR (-g -O0)

```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg);
6   for (; i < MAX; ++i)
7   {
8     out1 ^= data[i];
9   }
10
11  for (i = tmp; i < MAX; ++i)
12  {
13    out2 &= data[i];
14  }
15  g(out1, out2);
16  return tmp;
17}
```

```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg);
6   for (; i < MAX; ++i)
7   {
8     out1 ^= data[i];
9   }
10
11  for (i = tmp; i < MAX; ++i)
12  {
13    out2 &= data[i];
14  }
15  g(out1, out2);
16  return tmp;
17}
```

```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg); int *p = &data[tmp];
6   for (; i < MAXp < &data[MAX]; ++ip)
7   {
8     out1 ^= data[i]*p;
9   }
10
11  for (i = tmpp = &data[tmp]; i < MAX
12       p<&data[MAX]; ++ip) {
13    out2 &= data[i]*p;
14  }
15  g(out1, out2);
16  return tmp;
17}
```

```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg); int *p = &data[tmp];
6   for (; i < MAXp < &data[MAX]; ++ip)
7   {
8     out1 ^= data[i]*p;
13    out2 &= data[i]*p;
9   }
11  for (i = tmp; i < MAX; ++i)
12  {
13    out2 &= data[i];
14  }
15  g(out1, out2);
16  return tmp;
17}
```

```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg);
6   for (; i < MAX; ++i)
7   {
8     out1 ^= data[i];
9   }
10
11  for (i = tmp; i < MAX; ++i)
12  {
13    out2 &= data[i];
14  }
15  g(out1, out2);
16  return tmp;
17}
```

`data:` in `r1` at all points

`arg:` in `r2` at all points

`i:` in `r3` from 3

`tmp:` in `r4` from 5

`out1:` in `r5` from 3

`out2:` in `r6` from 3

```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg);
6   for (; i < MAX; ++i)
7   {
8     out1 ^= data[i];
9   }
10
11  for (i = tmp; i < MAX; ++i)
12  {
13    out2 &= data[i];
14  }
15  g(out1, out2);
16  return tmp;
17}
```

data: in r1 at all points

arg: in r2 at all points

i: value 0 from 3 to 5 in r3 from 5

tmp: in r4 from 5

out1: in r5 from 3

out2: in r6 from 3

```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg); int *p = &data[tmp];
6   for (; i < MAXp < &data[MAX]; ++ip)
7   {
8     out1 ^= data[i]*p;
9   }
10
11  for (i = tmpp = &data[tmp]; i < MAX
12       p<&data[MAX]; ++ip) {
13    out2 &= data[i]*p;
14  }
15  g(out1, out2);
16  return tmp;
17}
```

data: in r1 at all points

arg: in r2 at all points

i: value 0 from 3 to 5
value (r7-r1)/4 from 5

tmp: in r4 from 5

out1: in r5 from 3

out2: in r6 from 3

```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg); int *p = &data[tmp];
6   for (; i < MAXp < &data[MAX]; ++ip)
7   {
8     out1 ^= data[i]*p;
13    out2 &= data[i]*p;
9   }
11  for (i = tmp; i < MAX; ++i)
12  {
13    out2 &= data[i];
14  }
15  g(out1, out2);
16  return tmp;
17}
```

data: in r1 at all points

arg: in r2 at all points

i: value 0 from 3 to 5
value (r7-r1)/4 from 5

tmp: in r4 from 5

out1: in r5 from 3

out2: in r6 from 3

Some difficulties defining correctness:

the object program may maintain different state (not just more state)
- a mapping may exist only ‘up to’ how state is observed

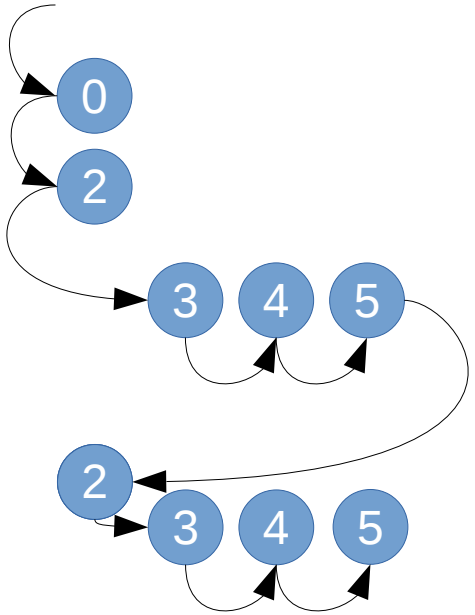
the object program may maintain less state
- state can be dropped if ‘not subsequently observed’

the object program does not simulate the source program!
- operations with no ‘observable’ effect can be deleted
- operations can be reordered, if the change is not ‘observable’

What notion of ‘observability’?
- program input/output (sequential semantics)
- memory model (concurrent semantics)
- separate compilation contract incl. ABI
- **NOT debugging! yet debugging ‘observes’ execution**

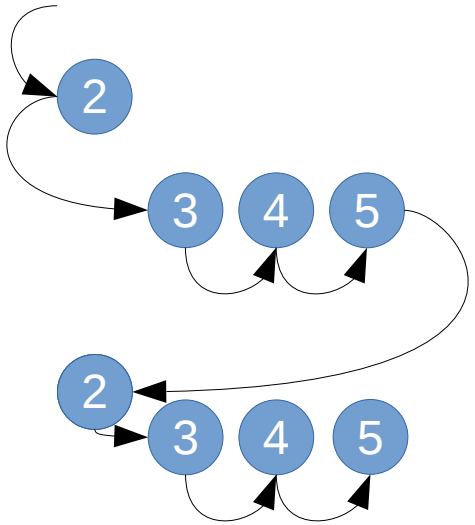
Suppose MAX is 5 and `get_start` returns 2.
Over such a program path, what does `i` do?

```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg);
6   for (; i < MAX; ++i)
7   {
8     out1 ^= data[i];
9   }
10
11  for (i = tmp; i < MAX; ++i)
12  {
13    out2 &= data[i];
14  }
15  g(out1, out2);
16  return tmp;
17}
```



Suppose MAX is 5 and `get_start` returns 2.
What does `r3` (holding `i` from line 5) do?

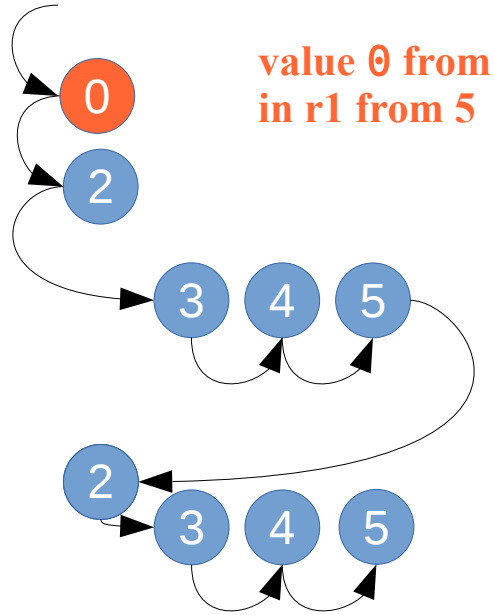
```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg);
6   for (; i < MAX; ++i)
7   {
8     out1 ^= data[i];
9   }
10
11  for (i = tmp; i < MAX; ++i)
12  {
13    out2 &= data[i];
14  }
15  g(out1, out2);
16  return tmp;
17}
```



Suppose MAX is 5 and `get_start` returns 2.

What does `i` do?

```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg);
6   for (; i < MAX; ++i)
7   {
8     out1 ^= data[i];
9   }
10
11  for (i = tmp; i < MAX; ++i)
12  {
13    out2 &= data[i];
14  }
15  g(out1, out2);
16  return tmp;
17}
```

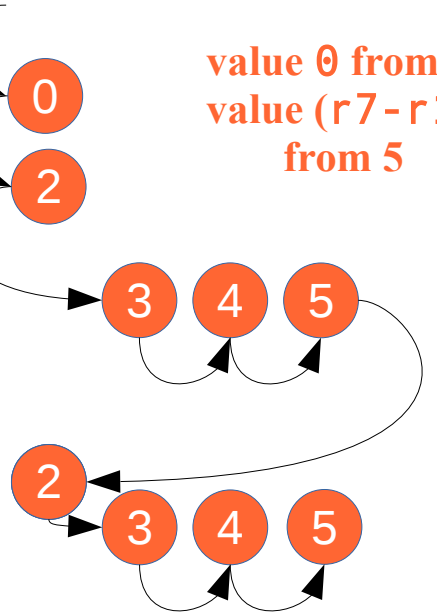


value `0` from 3 to 5
in r1 from 5

Suppose MAX is 5 and `get_start` returns 2.

What does `i` do?

```
1  int f(int *data, void *arg)
2  {
3    int i = 0, tmp, out1 = 0, out2 = 0;
4
5    i = tmp = get_start(arg); int *p = &data[tmp];
6    for (; i < MAXp < &data[MAX]; ++ip)
7    {
8      out1 ^= data[i]*p;
9    }
10
11   for (i = tmpp = &data[tmp]; i < MAX
12         p<&data[MAX]; ++ip) {
13     out2 &= data[i]*p;
14   }
15   g(out1, out2);
16   return tmp;
17 }
```

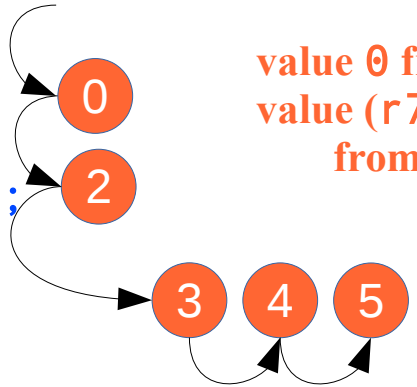


value `0` from 3 to 5
value `(r7-r1)/4` from 5

Suppose MAX is 5 and `get_start` returns 2.

What does `i` do?

```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg); int *p = &data[tmp];
6   for (; i < MAXp < &data[MAX]; ++ip)
7   {
8     out1 ^= data[i]*p;
13    out2 &= data[i]*p;
9   }
11  for (i = tmp; i < MAX; ++i)
12  {
13    out2 &= data[i];
14  }
15  g(out1, out2);
16  return tmp;
17}
```



value `0` from 3 to 5
value `(r7-r1)/4` from 5

Fusing the loops broke our property over i!

Deletion of operations and state (avoid saying “code”) is usually OK
- ... i.e. OK if we can residualise what was deleted
- depends on our debug format!
- in DWARF:
- can augment program state with anything *functionally redundant*
- can augment control flow with any straight-line spliceable segments
    -- move program counter and any local variable through arbitrary ‘virtual state’ sequence that can be computed from the preceding actual state
- can’t augment with state that the object program overwrites/destroys
- can’t augment with both-ways-live branches

Reordering is usually OK.
- our property explicitly accommodates it
- problem: reordering that affects *a variable with respect to itself*

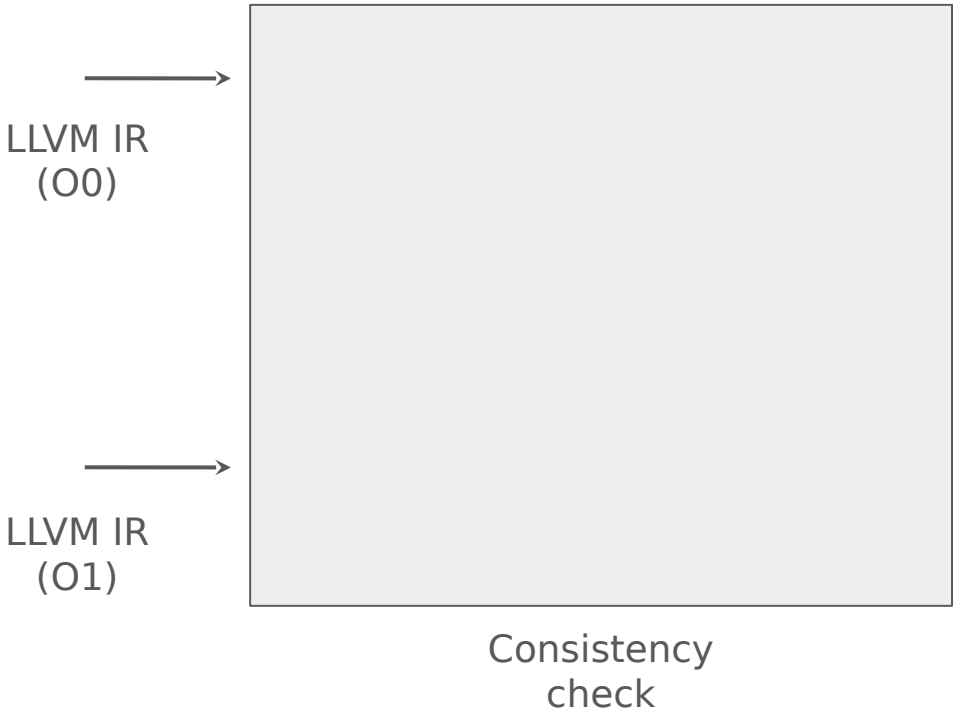
Fusing the loops broke our property over i!

We’re going to need a relaxation for these cases.
- hypothesis: few enough passes need this that is can be an annotation in the pass itself, passing through to the debug info

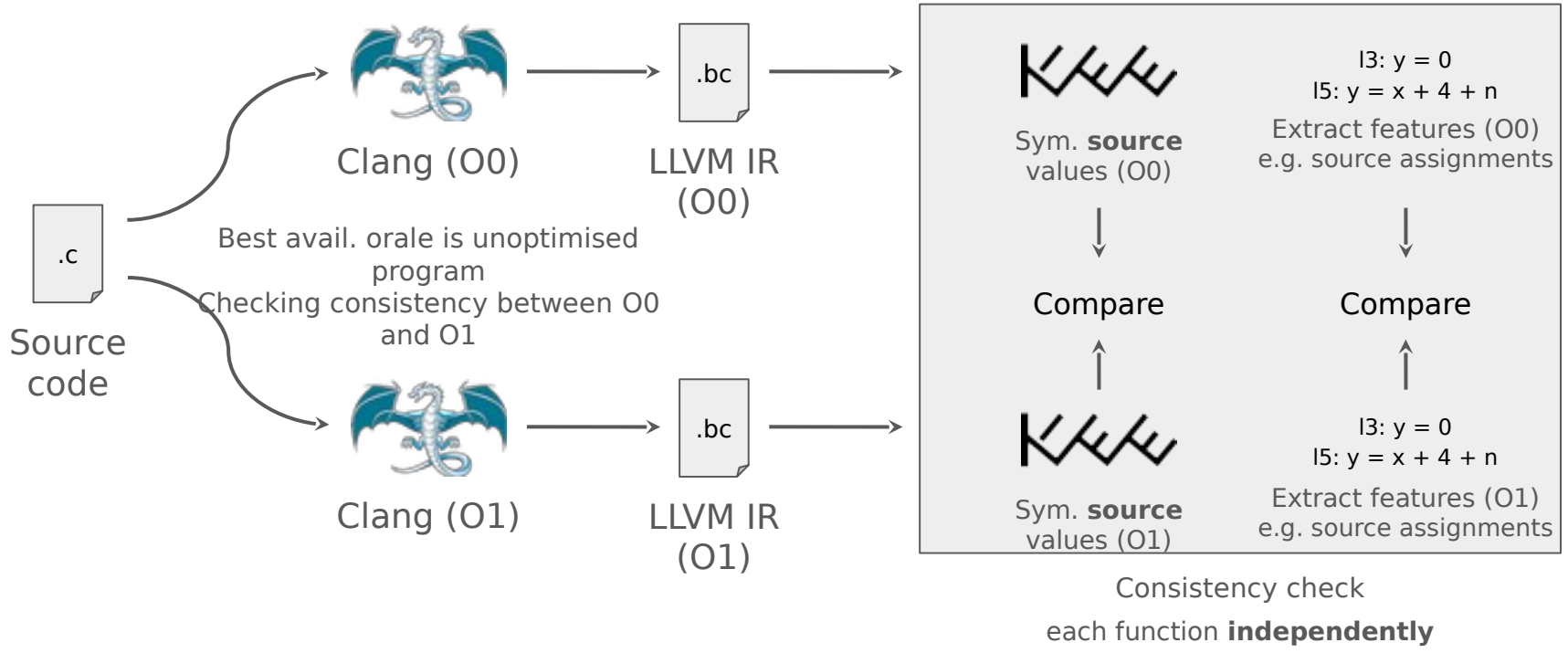
Possible relaxations:
- lifetime splitting: “i epoch 1”, “i epoch 2”
  ... when we fuse the later updates to i in among earlier updates to i, first rename i (effectively; not a user-visible rename, just checker-visible)

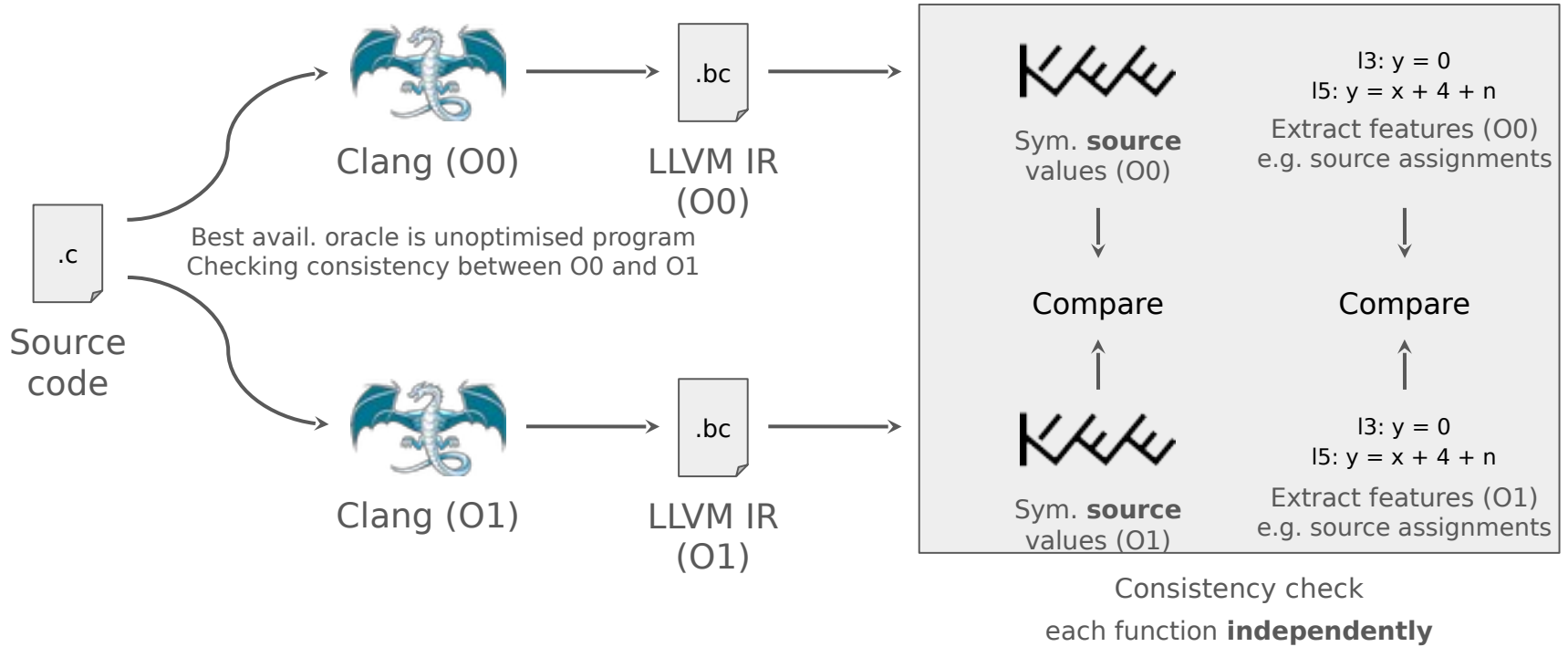
- “bag relaxation”: discard order, just treat line-annotated updates as a set (or bag, more properly)
 ... this is the extreme version of lifetime splitting; not clear it’s needed
 ... maybe needed for loop tiling: index variables progress in a different sequence (hmm, can factor in terms of chopping up a range?)

- “subsequence relaxation”: allow skipping ahead


LLVM IR
(O0)
LLVM IR
(O1)
Consistency
check

Debug info consistency check




.c
Source code
Clang (O0)
.bc
LLVM IR (O0)
Best avail. oracle is unoptimised program
Checking consistency between O0 and O1
Clang (O1)
.bc
LLVM IR (O1)
Sym. source values (O0)
Extract features (O0) e.g. source assignments
I3: y = 0
I5: y = x + 4 + n
Compare
Compare
Sym. source values (O1)
Extract features (O1) e.g. source assignments
I3: y = 0
I5: y = x + 4 + n
Consistency check
each function independently

Debug info consistency check

- Approach allows for code motion
  - Each variable is explored separately
  - Optimisations remain free to reorder instructions affecting different source variables
- Source variables values can be checked even if eliminated at run-time
  - Value assignment may be actual (run-time) or residual (debug-time) in different program versions
  - Can still check values as expected via formulae collected by KLEE at assignment points

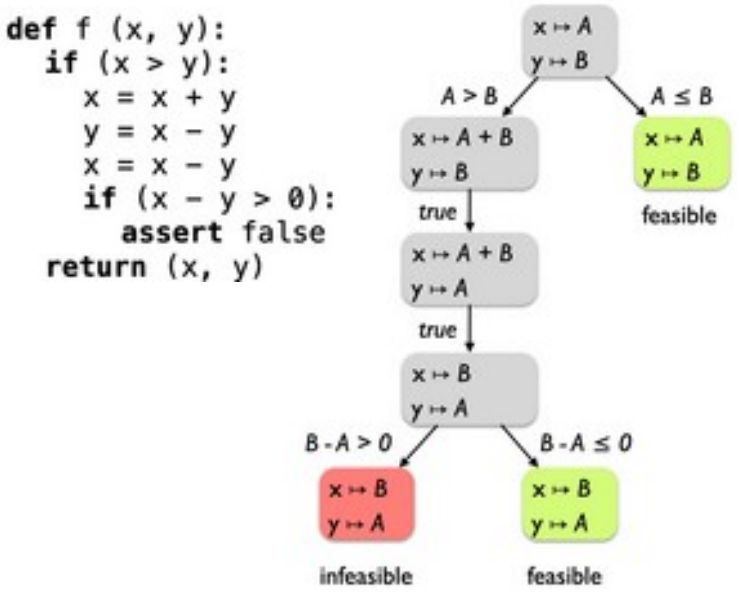
Ways of thinking about debug info

- If a variable is eliminated, it is not necessarily the case that it is absent from the debug illusion: some debug info formats can describe how to reconstruct it from state that remains
  - DWARF supports expressions in an interpreted stack machine language that the debugger can use to compute functions of program state
- The more thoroughly a variable is “eliminated” from the emitted program, the more it needs to be described in the debug info
- Rather than viewing optimisations and debugging as mutually excluding, it is more accurate to see debug info as **residualising** the eliminations or simplifications made during optimisation
  - Run-time program gets shorter during optimisation, debug info grows to maintain illusion

Symbolic execution

- Symbolic execution tools (such as KLEE) run a program using abstract formulae in place of concrete input values
- When branches are encountered, execution state is “forked” for each path
- Path constraints track requirements of each side of the conditional
- Can obtain symbolic formulae for program variables

```
def f (x, y):
  if (x > y):
    x = x + y
    y = x - y
    x = x - y
    if (x - y > 0):
      assert false
  return (x, y)
```



Images from 2016 lecture by Emina Torlak
https://courses.cs.washington.edu/courses/cse403/16au/lectures/L16.pdf

KLEE aimed at debug info

- KLEE can help us check debug info **more systematically**
- KLEE explores paths through LLVM IR automatically
- Symbolic IR values are evaluated during KLEE's program execution
- LLVM IR supports debug info mappings from IR to source values

```
define i32 @example(i32 %n)
  %mul = shl i32 %n, 1, !2 !c13
    ① KLEE tracks %mul as the symbolic value (Shl n 1) (simplified KQuery syntax)
  @dbg.value(i32 %mul, "x" !2)
    ② Debug info mapping states that source var x = IR value %mul
    ③ From ① and ②, it follows that x = symbolic value (Shl n 1)
```

Debug info example in abbreviated LLVM IR

Debug info consistency check

1. *Separately for each local source variable*, statically compute a pairwise matching of the IR-level assignment events that affect it in the two compilations of the program
    - Assignments are paired if they are “the same event” as identified by their source coordinates in debug info
2. Dynamically explore paths (via our extended KLEE) through the function and collect formulae for the IR values “assigned” at each identified assignment operation
3. For each source variable, for each pair of matched assignments to that variable, we use the SMT solver to check that the assigned formulae are equivalent

Assignment events in IR

- SSA value mapped directly to a source variable

```
%add = add 1, 2
@dbg.value(%add, "x")
```

- Other direct representations of a source variable

```
@dbg.value(3, "x")
```

Assignment events in IR

- Residualised source variables

```
%y = load %y*
@dbg.value(%y, "x", (add 1 %y))
```

- Stores to local memory allocations

```
%x* = alloca
@dbg.declare(%x*, "x")
store 3, %x*
```

Examples

Consistency check examples

- Example 1: inconsistency found
  - Unoptimised LLVM IR (O0)
  - Optimised LLVM IR (O1)

Example 1: inconsistency found

```
1 int example(int n) {
2   int x = n * 2;
3   int y = 0;
4   for (unsigned int i = 0; i < n; i++) {
5     y += x + 4 + n;
6   }
7   return y;
8 }
```

Source code

```
define i32 @example(i32 %n) {
entry:
  %y = alloca i32  ①  allocates stack space, %y points to this
storage
  @dbg.declare(i32* %y, "y" !3)  ②  source var y is stored at %y
  store i32 0, i32* %y, !3
    ③  stores constant (0) for source var y

for.body:
  %3 = load i32, i32* %x, !5
  %add = add i32 %3, 4, !5
  %4 = load i32, i32* %n.addr, !5
  %add1 = add i32 %add, %4, !5
  %5 = load i32, i32* %y, !5
  %add2 = add i32 %5, %add1, !5
  store i32 %add2, i32* %y, !5
    ④  stores %add2 for source var y
```

Unoptimised LLVM IR (O0)

Example 1: inconsistency found

```
1 int example(int n) {
2   int x = n * 2;
3   int y = 0;
4   for (unsigned int i = 0; i < n; i++) {
5     y += x + 4 + n;
6   }
7   return y;
8 }
```

Source code

```
define i32 @example(i32 %n) {
entry:
  %y = alloca i32  ①  allocates stack space, %y points to this
storage
  @dbg.declare(i32* %y, "y" !3)  ②  source var y is stored at %y
  store i32 0, i32* %y, !3
    ③  stores constant (0) for source var y

for.body:
  %3 = load i32, i32* %x, !5
  %add = add i32 %3, 4, !5
  %4 = load i32, i32* %n.addr, !5
  %add1 = add i32 %add, %4, !5
  %5 = load i32, i32* %y, !5
  %add2 = add i32 %5, %add1, !5
  store i32 %add2, i32* %y, !5
    ④  stores %add2 for source var y
```

At source line 3:
y = 0

At source line 5:
y = (Add 4 (Add
 (Mul 2 n) n))

Unoptimised LLVM IR (O0)

Example 1: inconsistency found

```
1 int example(int n) {
2   int x = n * 2;
3   int y = 0;
4   for (unsigned int i = 0; i < n; i++) {
5     y += x + 4 + n;
6   }
7   return y;
8 }
```

Source code

```
define i32 @example(i32 %n) {
entry:
  @dbg.value(i32 0, "y" !3)
    ① source var y = constant (0)
for.cond.cleanup.loopexit:
  %0 = add i32 %n, -1, !4
  %add = add i32 %n, 4
  %mul = shl i32 %n, 1, !2
  %add1 = add i32 %add, %mul
  %1 = mul i32 %0, %add1, !4
  %2 = mul i32 %n, 3, !4
  %3 = add i32 %1, %2, !4
  %4 = add i32 %3, 4, !4
    ② should be mapped to y, but debug mapping lost!
  @dbg.value(i32 undef, "y" !3)
    ③ dead debug mapping without an input value
```

Optimised LLVM IR (O1)

Example 1: inconsistency found

```
1 int example(int n) {
2   int x = n * 2;
3   int y = 0;
4   for (unsigned int i = 0; i < n; i++) {
5     y += x + 4 + n;
6   }
7   return y;
8 }
```

Source code

```
define i32 @example(i32 %n) {
entry:
  @dbg.value(i32 0, "y" !3)
    ① source var y = constant (0)
for.cond.cleanup.loopexit:
  %0 = add i32 %n, -1, !4
  %add = add i32 %n, 4
  %mul = shl i32 %n, 1, !2
  %add1 = add i32 %add, %mul
  %1 = mul i32 %0, %add1, !4
  %2 = mul i32 %n, 3, !4
  %3 = add i32 %1, %2, !4
  %4 = add i32 %3, 4, !4
    ② should be mapped to y, but debug mapping lost!
  @dbg.value(i32 undef, "y" !3)
    ③ dead debug mapping without an input value
```

At source line 3:
y = 0

Value mapping lost, should be:
y = %4 = (Add 4
 (Add
  (Mul (Add -1 n)
   (Add 4
    (Add n (Shl n 1))))
  (Mul 3 n)))

Optimised LLVM IR (O1)

Example 1: inconsistency found

```
define i32 @example(i32 %n) {
entry:
  %y = alloca i32  ①  allocates stack space, %y points to this
storage
  @dbg.declare(i32* %y, "y" !3)  ②  source var y is stored at %y
  store i32 0, i32* %y, !3
    ③  stores constant (0) for source var y

for.body:
  %3 = load i32, i32* %x, !5
  %add = add i32 %3, 4, !5
  %4 = load i32, i32* %n.addr, !5
  %add1 = add i32 %add, %4, !5
  %5 = load i32, i32* %y, !5
  %add2 = add i32 %5, %add1, !5
  store i32 %add2, i32* %y, !5
    ④  stores %add2 for source var y
```

At source line 3:
y = 0

At source line 5:
y = (Add 4 (Add
 (Mul 2 n) n))

Unoptimised LLVM IR (O0)

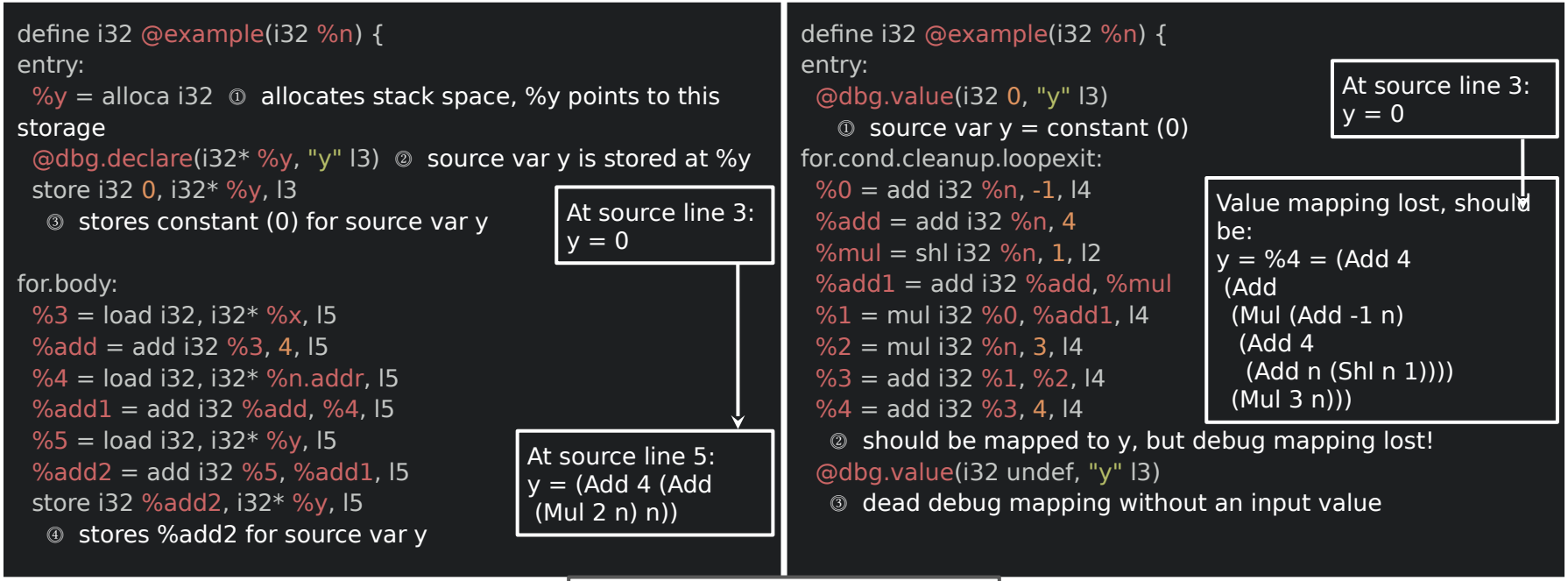
```
define i32 @example(i32 %n) {
entry:
  @dbg.value(i32 0, "y" !3)
    ①  source var y = constant (0)
for.cond.cleanup.loopexit:
  %0 = add i32 %n, -1, !4
  %add = add i32 %n, 4
  %mul = shl i32 %n, 1, !2
  %add1 = add i32 %add, %mul
  %1 = mul i32 %0, %add1, !4
  %2 = mul i32 %n, 3, !4
  %3 = add i32 %1, %2, !4
  %4 = add i32 %3, 4, !4
    ②  should be mapped to y, but debug mapping lost!
  @dbg.value(i32 undef, "y" !3)
    ③  dead debug mapping without an input value
```

At source line 3:
y = 0

Value mapping lost, should
be:
y = %4 = (Add 4
 (Add
  (Mul (Add -1 n)
   (Add 4
    (Add n (Shl n 1))))
  (Mul 3 n)))

Optimised LLVM IR (O1)

Example 1: inconsistency found



Unoptimised LLVM IR (O0)

Assignments: wrong source line
Values: mapping lost
Inconsistency found!

Optimised LLVM IR (O1)

Status

Current status

- Approach implemented in new tool built on top of KLEE
- Expects two LLVM modules (*.bc or *.ll), one before and one after optimisation

```
$ check-debug-info example-O0.ll example-O1.ll
```

- Produces consistency report for each function (via independent mode)

```
### Variables

✖ Value produced for `i` (decl src ln 4): missing line info
✖ Value produced for `y` (decl src ln 3): missing line info
~ Multiple assignments to `x` in range from ln 2, col 0 to ln 2, col 13
✔ 4 before variables found, 4 after variables found, 0 mismatched

### Assignments

✔ Before live range coverage

### Symbolic values

~ After `i` ln 4, col 0 coords don't match before ln 4, col 36
~ After `y` ln 3, col 0 coords don't match before ln 5, col 7
✖ After `y` ln 3, col 0 symbolic value doesn't match before ln 5, col 7
(Eq (Add w32 0x4
           (Add w32 (Mul w32 0x2
                            N0:(ReadLSB w32 0x0 n))
                    N0))
    0x0)
✖ Before symbolic values checked against after
```

Results

- Confirmed existing compiler debug issues to build confidence
  - 16 issues confirmed so far
- Discovered and filed new issues via real and generated programs
  - 9 issues discovered so far via Csmith-generated programs (1), examining Git (7), and author-written (1) code
  - Out of these 9 issues
    - 1 issue resolved in a later version
    - 7 issues confirmed as valid by LLVM community
    - 1 issue newly filed and awaiting confirmation

False positives

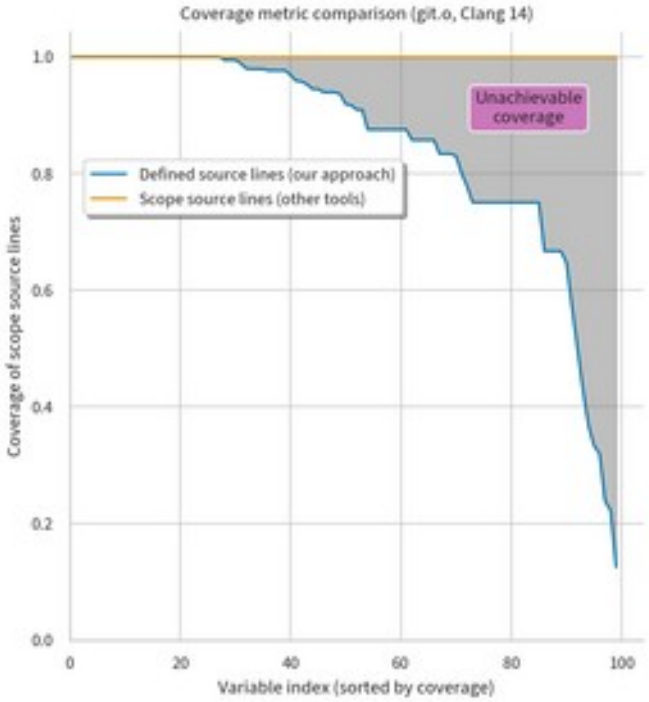
- Static assignment matching may be confused by dead code removal
  - Currently using a basic local analysis to detect obvious cases of this
  - May add more robustness by e.g. running LLVM’s own dead code removal passes
  - Could move toward fully dynamic approach
- Control flow changes may cause assignment to occur on subset of paths
  - Formulae presented to SMT solver unlikely to be equivalent in such cases

IR design obstacles

- LLVM debug info IR doesn’t explicitly state assignment locations
  - Location defined by next real instruction, but this may change during optimisation
  - Try to allow for slight changes by modelling assignment “generations” rather than requiring exact line number matches
  - May propose debug info IR changes to preserve more explicit locations

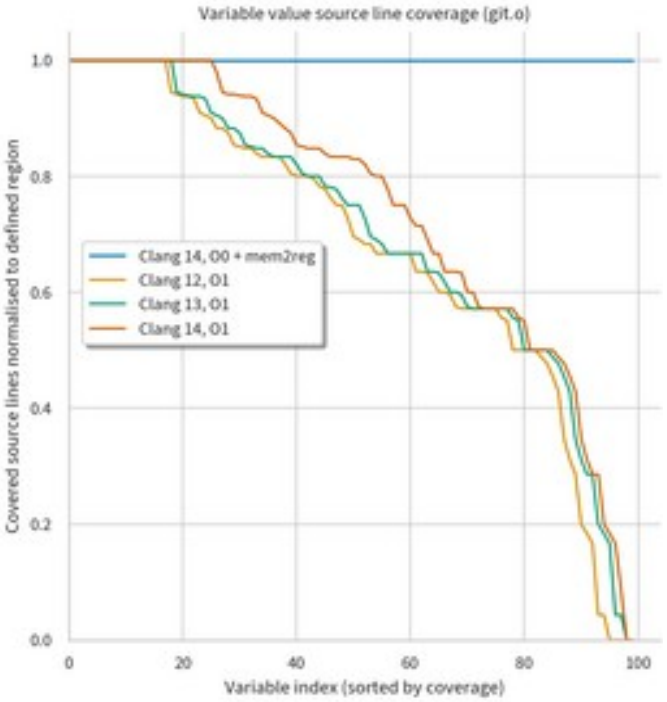
Metrics

- Other tools (llvm-dwarfdump, debuginfo-quality) measure coverage using parent scope which includes potentially undefined program points
- Our approach starts tracking coverage from point(s) of first definition
- Coverage in terms of source lines instead of instruction bytes
- Unlike past metrics, full coverage is actually attainable with our approach



Metrics

- Variable value coverage slowly but surely increasing with each new Clang version
- A large chasm remains uncovered
- Lots of room for future debug info coverage improvements



Future work

- Testing approaches
  - Gather debuggability stats by optimisation pass
  - Expand coverage to include machine code gen phase
- Formal verification
  - Ensure debug info is correct through approaches like translation validation, functional verification, etc.
  - Design compilers for “correct by construction” debug info residualisation
- Debug illusion
  - Extend debuggers to allow retaining more program state
  - Explore limits of debug info formats vs. “knowable” values from program state

Summary

- Debug info often gets lost during optimisation
- Prior testing approaches are often manual and checks values with heuristics
- Our approach enables strong consistency checks of debug value correctness by lifting symbolic formulae up to source variable values
- Testing tool implemented as research prototype, available soon
- Metrics tool ready for general use, also available soon

Thanks

J. Ryan Stinnett	!	Stephen Kell
convolv.es		humprog.org
King’s College London		King’s College London

Unusual application of symbolic execution

- Each function explored independently
- Each basic block visited at least once (similar to a compiler)
- Sufficient to gather generalised symbolic values for each source variable assignment from both target programs

Very different needs from
most applications of symbolic execution!

Priority of debug info for compiler authors

- Passes do try to preserve debug info…
  - e.g. LLVM's How to update debug info guide for optimisation pass authors
- Incentives not aligned for correct and complete debug info
  - Extra work to produce debug info on top of fast, correct run-time code
- No standard metrics for comparing debug info quality
  - Our own metric tracking coverage over variable's defined range (instead of scope) may help move the conversation forward here

Example of discovered issue

```c
time_t example(struct tm *tm) {
  int b[] = {9, 4};
  int c = tm->tm_year, d = tm->tm_mon;
  int day = tm->tm_mday;
  day--;
  return c + b[d] + day;
}
```

Source code

```
define i64 @example(ptr readonly %tm) {
entry:
  ...
  @dbg.value(i32 %2, "day")
  %dec = add i32 %2, -1
  @dbg.value(i32 %dec, "day")
  %idxprom = sext i32 %1 to i64
  %arrayidx = getelementptr [2 x i32], ptr @b,
              i64 0, i64 %idxprom
  %3 = load i32, ptr %arrayidx
  %add = add i32 %0, %3
  %add1 = add i32 %add, %dec
  %conv = sext i32 %add1 to i64
  ret i64 %conv
}
```

Before Reassociate pass (O1)

Example of discovered issue

```
time_t example(struct tm *tm) {
  int b[] = {9, 4};
  int c = tm->tm_year, d = tm->tm_mon;
  int day = tm->tm_mday;
  day--;
  return c + b[d] + day;
}
```

Source code

```
define i64 @example(ptr readonly %tm) {
entry:
  ...
  @dbg.value(i32 %2, "day")
  @dbg.value(i32 poison, "day")
  %idxprom = sext i32 %1 to i64
  %arrayidx = getelementptr [2 x i32], ptr @b,
              i64 0, i64 %idxprom
  %3 = load i32, ptr %arrayidx
  %add = add i32 %0, -1
  %dec = add i32 %add, %2
  %add1 = add i32 %dec, %3
  %conv = sext i32 %add1 to i64
  ret i64 %conv
}
```

After Reassociate pass (O1)

Variable locations in DWARF

DWARF debug info generated by compiler (which we want to test) describes source variables via Turing-powerful stack machine with registers and memory as inputs

```
DW_TAG_variable
  DW_AT_name      ("y")

  DW_AT_decl_line  (3)
  DW_AT_type      (0x000000d5 "int")
  DW_AT_location
    [0x3f74, 0x3f7d):
      DW_OP_fbreg -12
    [0x3f7d, 0x3f90):
      <no location emitted>
    [0x3f90, 0x3f94):
      DW_OP_breg5 RDI+0, DW_OP_constu 0xffffffff, DW_OP_and,
      DW_OP_lit1, DW_OP_shl, DW_OP_stack_value
```

Simulated output illustrating expressivity of DWARF locations

Similar stack machine value expressions also appear in LLVM IR debug mappings

Locations are like a **symbolic mapping** of source variables to storage… Using KLEE's **symbolic values** from execution, we can ask an SMT solver to check the values in these locations!

Example 2: consistent

```
1 int example(int n) {
2   int x = n * 2;
3   int y = 0;
4   for (unsigned int i = 0; i < n; i++) {
5     y += x + 4 + n;
6   }
7   return y;
8 }
```

Source code

```
define i32 @example(i32 %n) {
entry:
  @dbg.value(i32 0, "y" !3)
    ① source var y = constant (0)
  %mul = shl i32 %n, 1, !2
  %add = add i32 %n, 4
  %add1 = add i32 %add, %mul

for.body:
  %y.011 = phi i32 [%add2, %for.body], [0, %entry]
  @dbg.value(i32 %y.011, "y" !3)
    ② source var y = %add2 or 0
  %add2 = add i32 %add1, %y.011, !5
  @dbg.value(i32 %add2, "y" !3)
    ③ source var y = %add2
```

Partially optimised LLVM IR (O1)

Example 2: consistent

```
1 int example(int n) {
2   int x = n * 2;
3   int y = 0;
4   for (unsigned int i = 0; i < n; i++) {
5     y += x + 4 + n;
6   }
7   return y;
8 }
```

Source code

```
define i32 @example(i32 %n) {
entry:
  @dbg.value(i32 0, "y" !3)
    ① source var y = constant (0)
  %mul = shl i32 %n, 1, !2
  %add = add i32 %n, 4
  %add1 = add i32 %add, %mul

for.body:
  %y.011 = phi i32 [%add2, %for.body], [0, %entry]
  @dbg.value(i32 %y.011, "y" !3)
    ② source var y = %add2 or 0
  %add2 = add i32 %add1, %y.011, !5
  @dbg.value(i32 %add2, "y" !3)
    ③ source var y = %add2
```

At source line 3:
y = 0

At source line 5:
y = (Add 4 (Add n (Mul 2 n)))

Partially optimised LLVM IR (O1)

Example 2: consistent

```
define i32 @example(i32 %n) {
entry:
  %y = alloca i32  ①  allocates stack space, %y points to this
storage
  @dbg.declare(i32* %y, "y" !3)  ②  source var y is stored at %y
  store i32 0, i32* %y, !3
    ③  stores constant (0) for source var y

for.body:
  %3 = load i32, i32* %x, !5
  %add = add i32 %3, 4, !5
  %4 = load i32, i32* %n.addr, !5
  %add1 = add i32 %add, %4, !5
  %5 = load i32, i32* %y, !5
  %add2 = add i32 %5, %add1, !5
  store i32 %add2, i32* %y, !5
    ④  stores %add2 for source var y
```

At source line 3:
y = 0

At source line 5:
y = (Add 4 (Add
 (Mul 2 n) n))

Unoptimised LLVM IR (O0)

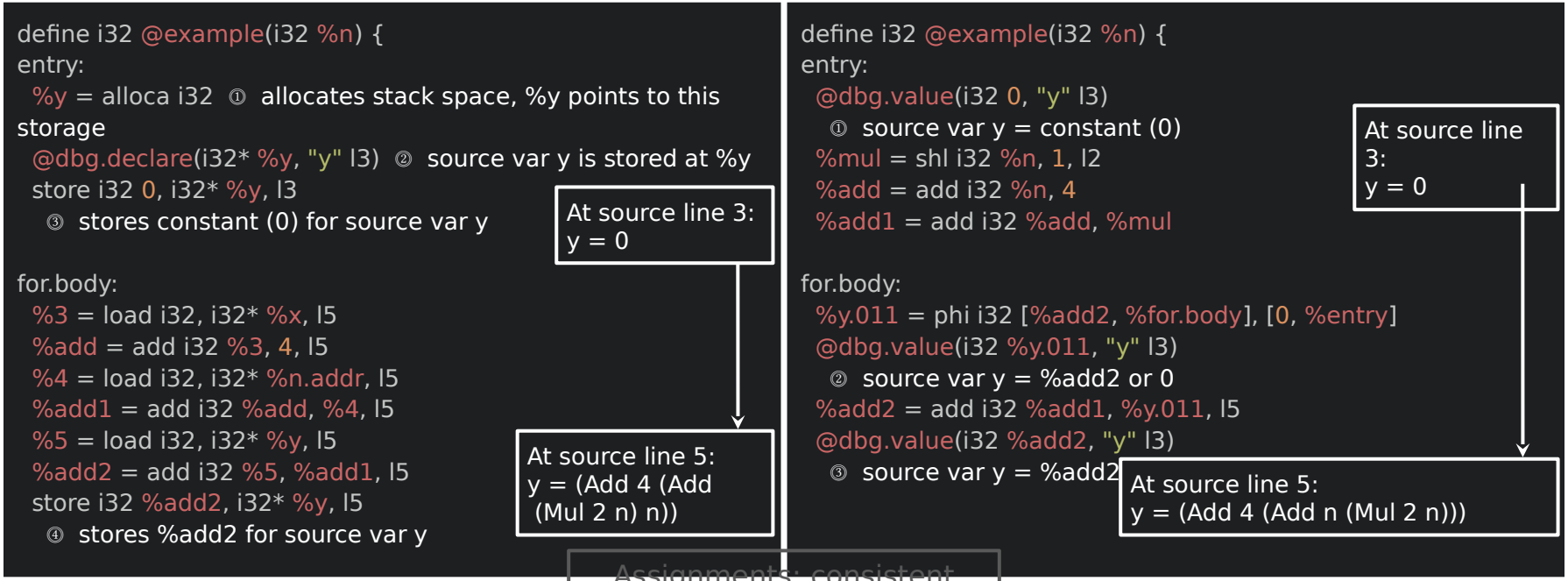
```
define i32 @example(i32 %n) {
entry:
  @dbg.value(i32 0, "y" !3)
    ①  source var y = constant (0)
  %mul = shl i32 %n, 1, !2
  %add = add i32 %n, 4
  %add1 = add i32 %add, %mul

for.body:
  %y.011 = phi i32 [%add2, %for.body], [0, %entry]
  @dbg.value(i32 %y.011, "y" !3)
    ②  source var y = %add2 or 0
  %add2 = add i32 %add1, %y.011, !5
  @dbg.value(i32 %add2, "y" !3)
    ③  source var y = %add2
```

At source line
3:
y = 0

At source line 5:
y = (Add 4 (Add n (Mul 2 n)))

Partially optimised LLVM IR (O1)

Example 2: consistent



Unoptimised LLVM IR (O0)

Assignments: consistent
Values: consistent
Debug info check passed!

Partially optimised LLVM IR (O1)

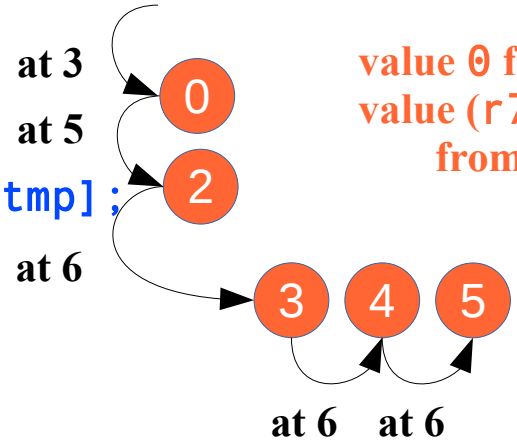
Consistency check examples

- Example 1: inconsistency found
  - Unoptimised LLVM IR (O0)
  - Optimised LLVM IR (O1)
  - Assignments: wrong source line
  - Values: mapping lost
- Example 2: consistent
  - Unoptimised LLVM IR (O0)
  - Partially optimised LLVM IR (O1, stopping early)
    - Optimisation stopped just before induction variable simplification
  - Assignments: consistent
  - Values: consistent

```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg); int *p = &data[tmp];
6   for (; i < MAXp < &data[MAX]; ++ip)
7   {
8     out1 ^= data[i]*p;
13    out2 &= data[i]*p;
9   }
11  for (i = tmp; i < MAX; ++i)
12  {
13    out2 &= data[i];
14  }
15  g(out1, out2);
16  return tmp;
17}
```

Suppose MAX is 5 and `get_start` returns 2.

What does `i` do?



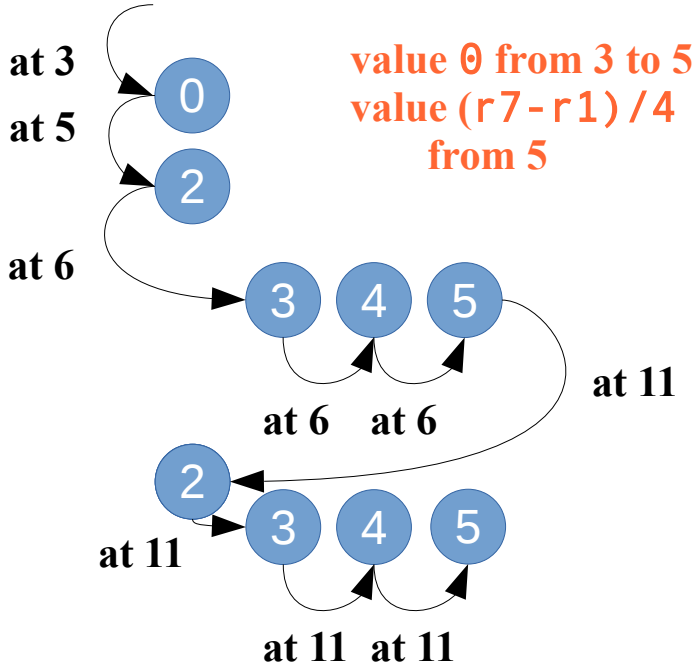
value `0` from 3 to 5
value `(r7-r1)/4` from 5

Set is:
{3:undef→0, 5:0→2, 6:2→3, 6:3→4, 6:4→5}

```
1  int f(int *data, void *arg)
2  {
3    int i = 0, tmp, out1 = 0, out2 = 0;
4
5    i = tmp = get_start(arg);
6    for (; i < MAX; ++i)
7    {
8      out1 ^= data[i];
9    }
10
11   for (i = tmp; i < MAX; ++i)
12   {
13     out2 &= data[i];
14   }
15   g(out1, out2);
16   return tmp;
17 }
```

Suppose MAX is 5 and `get_start` returns 2.

What does `i` do?

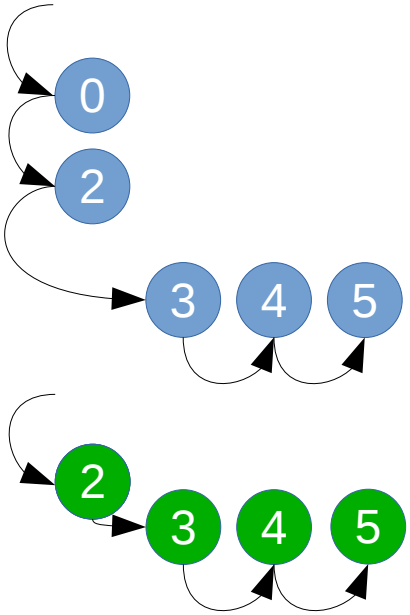


Set is:

{3:undef→0, 5:0→2, 6:2→3, 6:3→4, 6:4→5,
11:5→2, 12:2→3, 12:3→4, 12:3→5}

```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg);
6   for (; i < MAX; ++i)
7   {
8     out1 ^= data[i];
9   }
10
11  for (int j = tmp; j < MAX; ++j)
12  {
13    out2 &= data[j];
14  }
15  g(out1, out2);
16  return tmp;
17}
```

```
1 int f(int *data, void *arg)
2 {
3   int i = 0, tmp, out1 = 0, out2 = 0;
4
5   i = tmp = get_start(arg);
6   for (; i < MAX; ++i)
7   {
8     out1 ^= data[i];
9   }
10
11  for (int j = tmp; j < MAX; ++j)
12  {
13    out2 &= data[j];
14  }
15  g(out1, out2);
16  return tmp;
17}
```

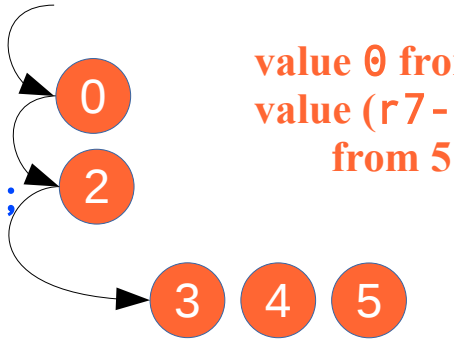


After fusion, `i` and `j` will be modelled by the same machine variable...

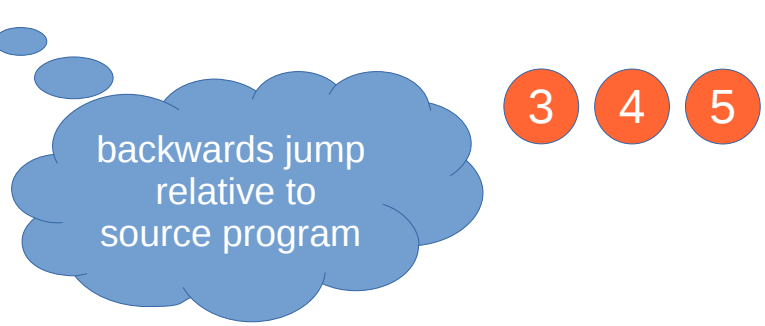
Suppose MAX is 5 and `get_start` returns 2.

What does `i` do?

```
1  int f(int *data, void *arg)
2  {
3    int i = 0, tmp, out1 = 0, out2 = 0;
4
5    i = tmp = get_start(arg); int *p = &data[tmp];
6    for (; i < MAXp < &data[MAX]; ++ip)
7    {
8      out1 ^= data[i]*p;
13     out2 &= data[i]*p;
9    }
11   for (i = tmp; i < MAX; ++i)
12   {
13     out2 &= data[i];
14   }
15   g(out1, out2);
16   return tmp;
17}
```



value 0 from 3 to 5
value (r7-r1)/4 from 5



Our cue to split `i` into two epochs
- for which our property is evaluated separately
- run concurrently (interleaved)...
- the same machine variable models both!